Exponential search

In computer science, an **exponential search** (also called **doubling search** or **galloping search** or **Struzik search**)[1] is an algorithm, created by Jon Bentley and Andrew Chi-Chih Yao in 1976, for searching sorted, unbounded/infinite lists.[2] There are numerous ways to implement this, with the most common being to determine a range that the search key resides in and performing a binary search within that range. This takes $O(\log i)$ time, where i is the position of the search key in the list, if the search key is in the list, or the position where the search key should be, if the search key is not in the list.

Exponential search

Class	Search algorithm
Data structure	Array
Worst-case performance	*O*(log *i*)
Best-case performance	*O*(1)
Average performance	*O*(log *i*)
Worst-case space complexity	*O*(1)
Optimal	Yes

Exponential search can also be used to search in bounded lists. Exponential search can even outperform more traditional searches for bounded lists, such as binary search, when the element being searched for is near the beginning of the array. This is because exponential search will run in $O(\log i)$ time, where i is the index of the element being searched for in the list, whereas binary search would run in $O(\log n)$ time, where n is the number of elements in the list.

Algorithm

Exponential search allows for searching through a sorted, unbounded list for a specified input value (the search "key"). The algorithm consists of two stages. The first stage determines a range in which the search key would reside if it were in the list. In the second stage, a binary search is performed on this range. In the first stage, assuming that the list is sorted in ascending order, the algorithm looks for the first exponent, j, where the value 2^j is greater than the search key. This value, 2^j becomes the upper bound for the binary search with the previous power of 2, 2^{j-1}, being the lower bound for the binary search.[3]

```
// Returns the position of key in the array arr of length size.
template <typename T>
int exponential_search(T arr[], int size, T key)
{
    if (size == 0) {
        return NOT_FOUND;
    }

    int bound = 1;
    while (bound < size && arr[bound] < key) {
        bound *= 2;
    }
```

```
    return binary_search(arr, key, bound/2, min(bound, size));
}
```

In each step, the algorithm compares the search key value with the key value at the current search index. If the element at the current index is smaller than the search key, the algorithm repeats, skipping to the next search index by doubling it, calculating the next power of 2.[3] If the element at the current index is larger than the search key, the algorithm now knows that the search key, if it is contained in the list at all, is located in the interval formed by the previous search index, 2^{j-1}, and the current search index, 2^j. The binary search is then performed with the result of either a failure, if the search key is not in the list, or the position of the search key in the list.

Performance

The first stage of the algorithm takes $O(\log i)$ time, where i is the index where the search key would be in the list. This is because, in determining the upper bound for the binary search, the while loop is executed exactly $\lceil \log(i) \rceil$ times. Since the list is sorted, after doubling the search index $\lceil \log(i) \rceil$ times, the algorithm will be at a search index that is greater than or equal to i as $2^{\lceil \log(i) \rceil} \geq i$. As such, the first stage of the algorithm takes $O(\log i)$ time.

The second part of the algorithm also takes $O(\log i)$ time. As the second stage is simply a binary search, it takes $O(\log n)$ where n is the size of the interval being searched. The size of this interval would be $2^j - 2^{j-1}$ where, as seen above, $j = \log i$. This means that the size of the interval being searched is $2^{\log i} - 2^{\log i - 1} = 2^{\log i - 1}$. This gives us a runtime of $\log(2^{\log i - 1}) = \log(i) - 1 = O(\log i)$.

This gives the algorithm a total runtime, calculated by summing the runtimes of the two stages, of $O(\log i) + O(\log i) = 2\,O(\log i) = O(\log i)$.

Alternatives

Bentley and Yao suggested several variations for exponential search.[2] These variations consist of performing a binary search, as opposed to a unary search, when determining the upper bound for the binary search in the second stage of the algorithm. This splits the first stage of the algorithm into two parts, making the algorithm a three-stage algorithm overall. The new first stage determines a value j', much like before, such that $2^{j'}$ is larger than the search key and $2^{j'/2}$ is lower than the search key. Previously, j' was determined in a unary fashion by calculating the next power of 2 (i.e., adding 1 to j). In the variation, it is proposed that j' is doubled instead (e.g., jumping from 2^2 to 2^4 as opposed to 2^3). The first j' such that $2^{j'}$ is greater than the search key forms a much rougher upper bound than before. Once this j' is found, the algorithm moves to its second stage and a binary search is performed on the interval formed by $j'/2$ and j', giving the more accurate upper bound exponent j. From here, the third stage of the algorithm performs the binary search on the interval 2^{j-1} and 2^j, as before. The performance of this variation is

$\lfloor \log i \rfloor + 2\lfloor \log(\lfloor \log i \rfloor + 1) \rfloor + 1 = O(\log i)$.

Bentley and Yao generalize this variation into one where any number, *k*, of binary searches are performed during the first stage of the algorithm, giving the *k*-nested binary search variation. The asymptotic runtime does not change for the variations, running in $O(\log i)$ time, as with the original exponential search algorithm.

Also, a data structure with a tight version of the dynamic finger property can be given when the above result of the *k*-nested binary search is used on a sorted array.[4] Using this, the number of comparisons done during a search is log (*d*) + log log (*d*) + ... + *O*(log $^*d$), where *d* is the difference in rank between the last element that was accessed and the current element being accessed.

Applications

An algorithm based on exponentially increasing the search band solves global pairwise alignment for $O(ns)$, where n is the length of the sequences and s is the edit distance between them.[5][6]

See also

- Linear search
- Binary search
- Interpolation search
- Ternary search
- Hash table

References

1. Baeza-Yates, Ricardo; Salinger, Alejandro (2010), "Fast intersection algorithms for sorted sequences", in Elomaa, Tapio; Mannila, Heikki; Orponen, Pekka (eds.), *Algorithms and Applications: Essays Dedicated to Esko Ukkonen on the Occasion of His 60th Birthday*, Lecture Notes in Computer Science, vol. 6060, Springer, pp. 45–61, Bibcode:2010LNCS.6060...45B (https://ui.adsabs.harvard.edu/abs/2010LNCS.6060...45B), doi:10.1007/978-3-642-12476-1_3 (https://doi.org/10.1007%2F978-3-642-12476-1_3), ISBN 9783642124754.
2. Bentley, Jon L.; Yao, Andrew C. (1976). "An almost optimal algorithm for unbounded searching". *Information Processing Letters*. **5** (3): 82–87. doi:10.1016/0020-0190(76)90071-5 (https://doi.org/10.1016%2F0020-0190%2876%2990071-5). ISSN 0020-0190 (https://search.worldcat.org/issn/0020-0190). OSTI 1318069 (https://www.osti.gov/biblio/1318069).
3. Jonsson, Håkan (2011-04-19). "Exponential Binary Search" (https://web.archive.org/web/20200601234125/https://sites.google.com/site/d7013e11/announcements/exponentialbinaryse

arch). Archived from the original (https://sites.google.com/site/d7013e11/announcements/exponentialbinarysearch) on 2020-06-01. Retrieved 2014-03-24.

4. Andersson, Arne; Thorup, Mikkel (2007). "Dynamic ordered sets with exponential search trees". *Journal of the ACM*. **54** (3): 13. arXiv:cs/0210006 (https://arxiv.org/abs/cs/0210006). doi:10.1145/1236457.1236460 (https://doi.org/10.1145%2F1236457.1236460). ISSN 0004-5411 (https://search.worldcat.org/issn/0004-5411). S2CID 8175703 (https://api.semanticscholar.org/CorpusID:8175703).
5. Ukkonen, Esko (March 1985). "Finding approximate patterns in strings" (https://dx.doi.org/10.1016/0196-6774(85)90023-9). *Journal of Algorithms*. **6** (1): 132–137. doi:10.1016/0196-6774(85)90023-9 (https://doi.org/10.1016%2F0196-6774%2885%2990023-9). ISSN 0196-6774 (https://search.worldcat.org/issn/0196-6774).
6. Šošić, Martin; Šikić, Mile (2017). "Edlib: A C/C ++ library for fast, exact sequence alignment using edit distance" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5408825). *Bioinformatics*. **33** (9): 1394–1395. bioRxiv 10.1101/070649 (https://doi.org/10.1101%2F070649). doi:10.1093/bioinformatics/btw753 (https://doi.org/10.1093%2Fbioinformatics%2Fbtw753). PMC 5408825 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5408825). PMID 28453688 (https://pubmed.ncbi.nlm.nih.gov/28453688).
